WEEK 10 INTERNSHIP PROJECT REPORT

Customer Churn Prediction — Data Preprocessing & Feature Engineering

Data Preprocessing & Feature Engineering

Submitted by

Sumit Gupta

Submission date: 10 October 2026

GitHub Repository

https://github.com/SumitGupta9627/Week10-Customer-Churn-Prediction

Table of Contents

1. Project Overview and Business Problem

2. Project Objectives

3. Dataset Description

4. Tools and Technologies

5. Data Preprocessing Pipeline

6. Encoding Techniques

7. Feature Scaling

8. Feature Engineering

9. Model Development and Leakage Prevention

10. Model Evaluation

11. Testing and Validation Evidence

12. Project Folder Structure

13. Setup and Installation

14. Screenshots and Visual Evidence

15. Challenges, Limitations, and Future Improvements

16. Requirement Compliance Review

17. Conclusion

18. GitHub Repository

Page numbers are included in the footer. Page positions may vary if the document is reflowed.

1. Project Overview and Business Problem

Customer churn occurs when a customer stops using a company's product or service. When churn is high, a business may lose recurring revenue and need to spend additional resources acquiring replacement customers. This project prepares customer data and builds a classification workflow to estimate whether a customer will churn.

The repository describes a learning-scale end-to-end workflow: data audit, feature engineering, categorical encoding, scaling, outlier analysis, preprocessing with scikit-learn, Random Forest training, evaluation, and validation checks. The output is a demonstration of a machine-learning process, not a production-ready retention system.

2. Project Objectives

- Audit the dataset and summarize its structure, data quality, and target distribution.
- Prepare numeric and categorical variables using reproducible preprocessing steps.
- Demonstrate three encoding approaches: One-Hot Encoding, Ordinal Encoding, and Label Encoding for target labels.
- Demonstrate StandardScaler and MinMaxScaler and explain their different purposes.
- Create and document eight engineered features based on existing columns.
- Train a Random Forest classifier with a stratified train/test split and a preprocessing pipeline.
- Evaluate the churn class using accuracy, precision, recall, F1-score, a classification report, and a confusion matrix.
- Record validation evidence and identify the limitations that should be addressed before real-world use.

3. Dataset Description

The repository's included file is churn_data.csv. The README and preprocessing report describe 500 records and 9 original columns. The available repository documentation does not identify an external download URL or establish that this is a standard public telecom dataset; therefore, the report treats churn_data.csv as the supplied project dataset and does not claim a specific external source.

Column	Type / role	Description
CustomerID	Identifier (text)	Customer reference; excluded from model inputs.
Tenure	Numeric	Customer tenure, in the dataset's tenure units (interpreted as months by the feature definitions).
MonthlyCharges	Numeric	Current monthly charge.
TotalCharges	Numeric	Accumulated charges; used in a derived historical-spend proxy.
Contract	Categorical nominal	Contract category, such as Month-to-month, One year, Two year.
PaymentMethod	Categorical nominal	Payment method category.
PaperlessBilling	Categorical nominal/binary	Paperless billing status.
SeniorCitizen	Binary numeric	0/1 indicator.
Churn	Binary target	1 = churn; 0 = retained.

Dataset audit reported: 500 rows, 9 columns, no missing values in the listed columns, 447 retained customers and 53 churned customers. The churn rate is 10.6% (53/500). The 80/20 stratified split contains 100 test records: 89 retained and 11 churned.

The listed numeric columns are Tenure, MonthlyCharges, TotalCharges, SeniorCitizen, and Churn; CustomerID and the service/contract/payment fields are categorical or identifier-like. Exact pandas dtypes should be confirmed by running df.info() in the local environment because the repository summary does not publish a dtype table.

4. Tools and Technologies

Tool	Use in this project
Python 3.10+	Programming language (repository setup guidance).
Pandas	Load, inspect, clean, and transform tabular data.
NumPy	Numerical operations and feature calculations.
scikit-learn	Imputers, encoders, scalers, ColumnTransformer, Pipeline, train/test split, Random Forest, and evaluation metrics.
Jupyter Notebook / IPykernel	Interactive development and execution of the project notebook.
Matplotlib	Visualization support and notebook charts.
GitHub	Version control and project/documentation hosting.

The committed requirements.txt lists pandas>=2.0, numpy>=1.24, matplotlib>=3.7, scikit-learn>=1.3, jupyter>=1.0, and ipykernel>=6.0. No separate dependency is listed for Seaborn or XGBoost; they are not claimed as required tools.

5. Data Preprocessing Pipeline

The project's documented pipeline is designed to keep learned preprocessing steps inside a scikit-learn Pipeline/ColumnTransformer. The train/test split is performed before fitting the final pipeline, so imputation statistics, encoders, and scaling parameters are learned from training data rather than the full dataset.

Step	Implementation documented	Reason
1. Load and audit	Read churn_data.csv; inspect shape, columns, missing values, and target counts.	Understand data quality before transformations.
2. Identifier handling	Exclude CustomerID from model features.	An identifier is not a meaningful behavioral signal and can encourage spurious patterns.
3. Missing-value handling	Numeric median imputation; categorical most-frequent imputation inside the pipeline.	Provides a defined fallback if new/inference data contains missing values, even though the audit found none in the supplied file.
4. Outlier review	IQR and Z-score checks on Tenure, MonthlyCharges, and TotalCharges; report counts rather than automatically deleting rows.	Potentially unusual charges or tenure may be valid customers; investigate before removal.
5. Feature creation	Create eight derived variables from existing input fields.	Represent tenure, charges, and contract/payment signals in forms that may help classification.
6. Encoding	One-Hot Encoding for nominal fields; Ordinal Encoding for ordered TenureBand; LabelEncoder demonstrated for target labels only.	Convert categories into model-compatible numeric representations while respecting category meaning.
7. Scaling	StandardScaler in the main pipeline; MinMaxScaler demonstrated separately.	Show two common approaches to numeric feature scaling.
8. Model fit	Fit preprocessing and Random Forest as a single pipeline on training data.	Keep training and inference transformations consistent and reduce leakage risk.

Duplicate handling: the public preprocessing report does not state a numeric duplicate count or a specific duplicate-removal action. Therefore, duplicate checks should be run and their output recorded before claiming that duplicates were removed or that none exist.

Missing values: the audit reports zero missing values in all nine columns. Median and most-frequent imputers are still included as robust pipeline components for possible future data. Their presence does not mean that missing values were found in the supplied CSV.

6. Encoding Techniques

Technique	How it works	Use in this project
One-Hot Encoding	Creates a separate binary indicator column for each category.	Nominal columns Contract, PaymentMethod, and PaperlessBilling; avoids imposing an artificial ranking.
Ordinal Encoding	Maps categories to integer values according to an explicitly defined order.	TenureBand: New, Established, Loyal. The order must be specified intentionally because the values imply ranking.
Label Encoding	Maps class labels to integer identifiers.	Demonstrated for the target labels only: churn/retained represented as 1/0. It should not be used to assign arbitrary numeric order to nominal input categories.

Unknown categories: OneHotEncoder is configured to handle categories not observed during training, and the validation evidence reports a check for unseen-category handling. This helps make the pipeline more robust when new customer records contain a previously unseen nominal value.

7. Feature Scaling

StandardScaler transforms a numeric feature by subtracting the training mean and dividing by the training standard deviation. The transformed training feature typically has a mean near 0 and a standard deviation near 1. It is the scaler used in the main model pipeline.

MinMaxScaler maps a feature to a selected range, commonly [0, 1], using the training minimum and maximum. It is demonstrated separately in the notebook. It can be useful when a bounded range is desired, but extreme values can strongly affect the transformed range.

The validation evidence records that the MinMaxScaler demonstration produced minimums of 0 and maximums of 1 for selected columns, and that StandardScaler produced means near 0 and standard deviations near 1. The report does not claim that both scalers are simultaneously applied to the same model inputs: StandardScaler is the documented main-pipeline scaler.

8. Feature Engineering

Engineered feature	Definition	Purpose / caution
TenureYears	Tenure / 12	Expresses tenure in years for easier interpretation.
EstimatedLifetimeValue	Current TotalCharges used as a proxy	Represents accumulated charges to date, not a forecast of future lifetime value.
AverageMonthlySpend	TotalCharges / Tenure; zero tenure guarded as missing	Approximates historical monthly spend; depends on the tenure unit and needs a defined zero-tenure fallback.
MonthlyChargeToAverageRatio	MonthlyCharges / AverageMonthlySpend	Compares the current charge with historical average spend; division by zero is guarded.
LongTenureFlag	1 if Tenure >= 24, otherwise 0	Captures a simple tenure threshold.
MonthToMonthFlag	1 if Contract is Month-to-month, otherwise 0	Represents a contract type that may be associated with churn.
ElectronicPaymentFlag	1 if PaymentMethod is Electronic Check, otherwise 0	Represents a payment-method indicator.
TenureBand	New (<=12 months), Established (13–36), Loyal (>36)	Groups tenure into ordered categories; exact boundaries should be checked against business context.

All eight features are documented as being derived from input columns rather than from Churn. CustomerID is excluded. Division by zero is handled by making the affected value missing so the numeric pipeline can impute it. These features describe available data; they do not establish causal reasons for churn.

9. Model Development and Leakage Prevention

The documented model is a RandomForestClassifier with 250 trees (n_estimators=250) and class_weight='balanced'. The data is split into 80% training and 20% testing using stratification and random_state=42. Stratification helps preserve the approximate target-class proportions in both subsets.

A ColumnTransformer applies numeric imputation and StandardScaler to numeric inputs, and categorical imputation plus One-Hot/Ordinal encoding to categorical inputs. The transformer and classifier are placed in a single scikit-learn Pipeline. This structure ensures that preprocessing is fitted on training data and then reused on test data, instead of fitting transformations independently on the test set.

Class weighting gives the minority churn class additional influence during model fitting. It does not add new churn records. Since the dataset contains only 53 churn cases, evaluation estimates—especially recall—are uncertain and should be validated using repeated stratified cross-validation and a larger, representative dataset.

10. Model Evaluation

Metric	Recorded result	Meaning
Accuracy	96.0%	96 of 100 test records were classified correctly.
Precision — churn class	100.0%	Of the customers predicted to churn, all were actual churn cases in this test split.
Recall — churn class	63.6%	The model identified about 7 of the 11 actual churn cases and missed the remainder.
F1-score — churn class	77.8%	Harmonic balance between churn precision and recall.

The recorded classification report lists retained (class 0): precision 0.96, recall 1.00, F1-score 0.98, support 89; churn (class 1): precision 1.00, recall 0.64, F1-score 0.78, support 11. Overall accuracy is 0.96 over 100 test records.

Confusion matrix values implied by the recorded test supports and rounded metrics are: 89 true negatives, 0 false positives, 4 false negatives, and 7 true positives. This is a reconstruction from the published classification report, not a substitute for the actual rendered notebook matrix. The screenshot of the matrix should be inserted after a local run if a visual artifact is required.

Interpretation: the reported model is conservative when predicting churn. Its 100% precision is encouraging for avoiding unnecessary retention outreach in this one split, but 63.6% recall means it misses a material share of churners. Accuracy alone is not sufficient because 89% of the test split is retained customers. These metrics are not a guarantee of future performance.

11. Testing and Validation Evidence

Check	Evidence status
Notebook executed top-to-bottom	Repository validation_evidence.md records completion without cell errors. Re-run locally before submission.
Dataset load / shape	Documented as 500 rows × 9 columns.
Eight engineered features	Documented as created.
MinMaxScaler range	Validation file records selected columns had minimum 0 and maximum 1.
StandardScaler statistics	Validation file records means near 0 and standard deviations near 1.
Train/test split and model fit	Documented as completed.
Prediction count assertion	Reported as passed in validation evidence.
Binary prediction-label assertion	Reported as passed in validation evidence.
Unseen-category handling assertion	Reported as passed in validation evidence.
Duplicate check count	Not stated in available report; run and record explicitly.
Fresh local environment / dependency installation	Not independently verified during this documentation review; run before final submission.

The validation file is evidence recorded by the project repository, not an independent re-execution in the reviewer's environment. The notebook should be restarted and run from top to bottom in the submission environment to confirm

reproducibility.

12. Project Folder Structure

Path	Purpose
churn_prediction_pipeline.ipynb	Main notebook: data inspection, encoding examples, feature engineering, outlier analysis, scaling, pipeline, model training/evaluation, and assertions.
churn_data.csv	Input dataset: 500 rows and 9 original columns, as documented.
README.md	Project summary, required features, run instructions, and high-level results.
preprocessing_report.md	Data audit, preprocessing choices, leakage prevention, model workflow, and limitations.
feature_engineering_documentation.md	Definitions and rationale for eight engineered features.
requirements.txt	Python package requirements for reproducing the notebook.
reports/model_evaluation_report.md	Recorded test split setup, metrics, classification report, interpretation, and limitations.
reports/validation_evidence.md	Recorded notebook execution and validation checks.
screenshots/	Repository folder described as containing dataset visualization images. The available listing does not establish that all requested execution/metrics/confusion-matrix screenshots are present.

The repository contains the three core files required by the internship brief: churn_prediction_pipeline.ipynb, churn_data.csv, and preprocessing_report.md. It also includes separate feature-engineering documentation and reports.

13. Setup and Installation

Recommended Windows / VS Code procedure:

```
# 1. Clone the repository
git clone https://github.com/SumitGupta9627/Week10-Customer-Churn-Prediction.git
cd Week10-Customer-Churn-Prediction

# 2. Create a virtual environment
python -m venv .venv

# 3. Activate it in PowerShell
.venv\Scripts\Activate.ps1

# 4. Install the repository dependencies
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# 5. Start Jupyter if needed
jupyter notebook
```

Open churn_prediction_pipeline.ipynb in VS Code or Jupyter. Select the Python interpreter/kernel from .venv, choose Restart Kernel and Run All (or Run All Cells), and check that every cell completes. Review the metrics and confusion matrix generated by the notebook. If PowerShell blocks activation, use Command Prompt and run .venv\Scripts\activate.bat.

Python 3.10 or newer is the repository's documented baseline. Use a clean environment to avoid dependency conflicts. The package versions are minimums rather than a fully pinned lockfile, so installations at different dates may resolve to different compatible versions.

14. Screenshots and Visual Evidence

The repository has a screenshots/ folder described in README as containing dataset visualization images. The web-accessible listing reviewed for this report did not expose the actual image contents, so this report does not embed or fabricate screenshots.

- Required before submission if the evaluator expects visual evidence: capture a screenshot of the notebook showing successful execution and the dataset shape.
- Capture the actual printed accuracy, precision, recall, F1-score, and classification report.

- Capture the actual confusion matrix generated by the notebook.
- Capture the GitHub repository root showing the required files and reports folder.

Insert the genuine images into this section or append them as an evidence appendix. Do not use a recreated chart or typed metric table as if it were a screenshot of program execution.

15. Challenges, Limitations, and Future Improvements

- Small dataset: 500 rows and only 53 churn cases; the test set has 11 churn cases, so one or two predictions materially affect recall.
- Class imbalance: 10.6% of records are churn cases. Accuracy can look high even if the model misses churners.
- Single split: reported results come from one stratified 80/20 split, not repeated cross-validation.
- Synthetic/unclear provenance: the repository does not establish an external source for the CSV in the reviewed documentation. Dataset provenance and collection methodology should be added.
- Limited predictors: only nine original columns are available. Support history, customer satisfaction, complaint data, and time-dependent trends are not represented.
- Feature semantics: EstimatedLifetimeValue is accumulated TotalCharges, not true future lifetime value. Feature names and thresholds should be reviewed with domain experts.
- Screenshots: requested runtime screenshots were not verified as available in the accessible repository listing.
- Future work: document the dataset source, validate duplicate counts, add repeated stratified cross-validation, compare against a simple baseline, tune the decision threshold for business costs, assess calibration, inspect feature importance responsibly, and test on a larger out-of-time dataset.

16. Requirement Compliance Review

Requirement	Status based on repository review	Action / note
Project overview and objectives	Covered	Included in this report.
Setup instructions	Covered	Clone, virtual environment, requirements install, and notebook run steps documented.
Code structure explanation	Covered	Important files and folders listed.
At least 3 encoding methods	Documented as implemented/demoed	One-Hot, Ordinal, LabelEncoder for target labels.
2 scaling techniques	Covered as demonstrations	StandardScaler in pipeline; MinMaxScaler separate demonstration.
5+ engineered features	Covered	Eight features documented.
Complete preprocessing pipeline with rationale	Covered at documentation level	Local rerun recommended to verify reproducibility.
Model evaluation metrics	Covered	Actual metrics taken from reports/model_evaluation_report.md.
Confusion matrix	Values inferable; actual visual not verified	Insert genuine notebook screenshot/output.
Screenshots of working application/notebook	Partially verified	Screenshots folder exists, but requested runtime screenshots were not verified; add genuine captures.
Duplicate check and count	Unverified in available report	Run explicit duplicate analysis and document result.
External dataset source	Not specified	Add the actual source URL or state clearly that the dataset was supplied/generated for the project.
Fresh local run	Not independently verified here	Restart and Run All in the final submission environment.

Overall assessment: the repository documents the core technical requirements, including three encoding approaches, two scaling demonstrations, eight engineered features, a preprocessing pipeline, a Random Forest model, evaluation metrics, and three validation assertions. However, visual evidence for the exact execution/metrics/confusion matrix, duplicate-check output, dataset provenance, and a fresh local rerun remain items to verify. The project should not be described as fully validated for production.

17. Conclusion

This project demonstrates a structured approach to preparing customer data for churn classification. It documents data auditing, categorical encoding, scaling, feature engineering, outlier review, a leakage-aware scikit-learn pipeline, Random Forest training, and classification evaluation. On the recorded 100-row test split, the model achieved 96.0% accuracy, 100.0% churn precision, 63.6% churn recall, and a 77.8% churn F1-score.

The main practical finding is that high accuracy does not mean all churners are detected. The model missed some churn cases, and the dataset is too small to support strong claims about future business performance. Before submission, rerun the notebook locally, add genuine screenshots, verify duplicates, and document the dataset source.

18. GitHub Repository

Repository: https://github.com/SumitGupta9627/Week10-Customer-Churn-Prediction

The repository is the source of the notebook, dataset, preprocessing report, feature engineering documentation, requirements, screenshots folder, evaluation report, and validation evidence.